



# An efficient multiple-kernel learning for pattern classification<sup>☆</sup>

Chi-Yuan Yeh, Wen-Pin Su, Shie-Jue Lee<sup>\*</sup>

Department of Electrical Engineering, National Sun Yat-Sen University, Kaohsiung 804, Taiwan, ROC

## ARTICLE INFO

### Keywords:

Support vector machines  
Multiple-kernel learning  
Semidefinite programming

## ABSTRACT

Support vector machines (SVMs) have been broadly applied to classification problems. However, a successful application of SVMs depends heavily on the determination of the right type and suitable hyperparameter settings of kernel functions. Recently, multiple-kernel learning (MKL) algorithms have been developed to deal with these issues by combining different kernels together. The weight with each kernel in the combination is obtained through learning. Lanckriet et al. proposed a way of deriving the weights by transforming the learning into a semidefinite programming (SDP) problem with a transduction setting. However, the amount of time and space required by this method is demanding. In this paper, we reformulate the SDP problem with an induction setting and incorporate two strategies to reduce the search complexity of the learning process, based on the comments discussed in the Lanckriet et al. paper. The primal and dual forms of SDP are derived. A discussion on computation complexity is given. Experimental results obtained from synthetic and benchmark datasets show that the proposed method runs efficiently in multiple-kernel learning.

© 2012 Elsevier Ltd. All rights reserved.

## 1. Introduction

Support vector machines (SVMs) have been shown to be an effective tool for solving classification problems (Cortes & Vapnik, 1995; Drucker, Wu, & Vapnik, 1999; Guo et al., 2000; Hou, Hsu, Lee, & Bystroff, 2003; Kecman, 2001; Khandoker, Palaniswami, & Karmakar, 2009; Kim, Jung, Park, & Kim, 2002; Pontil & Verri, 1998; Sloin & Burshtein, 2008; Ward, McGuffin, Buxton, & Jones, 2003; Wang, 2005). In using SVMs, it is not necessary to know explicitly the mapping function to be used. Instead, the inner product of two mapped vectors in the high-dimensional feature space is replaced with a kernel function. However, a successful application of SVMs depends heavily on the determination of the right type and suitable hyperparameter settings of kernel functions. Usually, one has to determine the kernel function type and the associated hyperparameter settings in advance. If kernel functions or hyperparameter settings are chosen inappropriately, they can lead to a poor performance. Therefore, choosing suitable kernel functions and hyperparameter settings becomes an important issue in the application of SVMs. Golub, Heath, and Wahba (1979) proposed a generalized cross-validation method for choosing a value of the regularization parameter from the data. Tsoi and Tan (1997) proposed a method to estimate the center and the spread of a radial

basis function (RBF) model based on some a priori information. Hsu et al. (2003) claimed that the RBF kernel is a reasonable first choice. The settings of the involved parameters are determined by cross-validation and grid-search. However, there are some situations where the RBF kernel is not suitable. In some situations the proposed procedure is not good enough, and other techniques such as feature selection may be needed. Most other researchers choose radial basis function kernels and use the trial and error heuristic or grid search to determine the settings of the hyperparameters (Chapelle, Vapnik, Bousquet, & Mukherjee, 2002; Duan, Keerthi, & Poo, 2003; Friedrichs & Igel, 2005; Kwok, 2000; Wang & Fu, 2005). This obviously takes a lot of efforts. In addition, using a single kernel may not be sufficient to solve a complex problem accurately. To deal with these difficulties, several researchers have adopted the multi-kernel approach in which multiple kernels are used to construct a combined kernel (Bach, Lanckriet, & Jordan, 2004; Bennett, Momma, & Embrechts, 2002; Crammer, Keshet, & Singer, 2003; Lanckriet, Cristianini, Bartlett, Ghaoui, & Jordan, 2004; Ong, Smola, & Williamson, 2005; Rakotomamonjy, Bach, Canu, & Grandvalet, 2007; Sonnenburg, Rätsch, Schäfer, & Schölkopf, 2006; Wang, Chen, & Sun, 2008; Yeh, Huang, & Lee, 2011).

The simplest way to combine multiple kernels is by averaging them. Having the participant kernels with the same weight may not be appropriate for the decision process. Therefore, the main issue of the multiple-kernel combination is to determine the optimal weight for each kernel. Lanckriet et al. (2004) used a linear combination of matrices to combine multiple kernels. They transformed multiple-kernel learning into a semidefinite programming (SDP)

<sup>☆</sup> This work was supported by the National Science Council under the grant NSC 95-2221-E-110-055-MY2, and by "Aim for the Top University Plan" of the National Sun Yat-Sen University and Ministry of Education.

<sup>\*</sup> Corresponding author.

E-mail address: [leesj@mail.ee.nsysu.edu.tw](mailto:leesj@mail.ee.nsysu.edu.tw) (S.-J. Lee).

problem, which, being convex, has a global optimum. SDP can then be solved efficiently by the interior-point method. Other multiple-kernel learning methods include those proposed by Bach et al. (2004), Sonnenburg et al. (2006), Rakotomamonjy et al. (2007), Rakotomamonjy, Bach, Canu, and Grandvalet (2008), Szafranski, Grandvalet, and Rakotomamonjy (2008), Gonen and Alpaydin (2008), Xu, Jin, King, and Lyu (2008). They deal with large-scale problems by iteratively using the SMO algorithm (Platt, 1999) to update the Lagrange multipliers and kernel weights in turn, i.e., Lagrange multipliers being updated with fixed kernel weights followed by kernel weights being updated with fixed Lagrange multipliers. Although these methods are faster than SDP, they tend to suffer from local minimum traps. Multiple-kernel learning methods based on hyperkernels have also been studied (Ong et al., 2005; Tsang & Kwok, 2006). Ong et al. (2005) introduced the method of hyperkernels with SDP, while Tsang and Kwok (2006) reformulated the problem as a second-order cone programming (SOCP) form. On the other hand, Crammer et al. (2003) and Bennett et al. (2002) used boosting methods to combine heterogeneous kernel matrices.

The multiple-kernel learning method proposed by Lanckriet et al. (2004) uses a transduction setting where the kernel matrix constructed from both the training and testing patterns is required to be positive semidefinite. The amount of time and space requirements grows rapidly as the number of patterns increases. We propose a method adopting an induction setting in which only the training patterns are used to construct the kernel matrix. Furthermore, two strategies are incorporated to reduce the search complexity of the learning process. One strategy requires the kernel weights to be non-negative, and the other restricts the sum of the weights to be one. Note that the induction setting and the non-negativity assumption were discussed in Lanckriet et al. (2004) as a way of simplifying the model development process. As a result, the amount of time and space requirements is greatly reduced. The primal and dual forms of SDP are derived. A discussion on computation complexity is given. Experimental results show that the proposed method runs efficiently in multiple-kernel learning.

The rest of this paper is organized as follows. Section 2 introduces basic concepts of SVMs. Section 3 briefly describes the multiple-kernel learning method proposed by Lanckriet et al. (2004). Section 4 presents our proposed method. Both primal and dual forms of SDP are derived. In Section 5, experimental results are given. Finally, we conclude this work in Section 6.

## 2. Soft margin SVMs

SVM is a kernel method for finding the maximum-margin hyperplane which separates the images of the input data into two groups in the feature space (Cortes & Vapnik, 1995; Cristianini & Shawe-Taylor, 2000; Schölkopf & Smola, 2001; Shawe-Taylor & Cristianini, 2004). To make the method more flexible and robust, all the patterns need not be correctly classified by the hyperplane. The misclassified patterns are penalized, and slack variables  $\xi_i$  are introduced to account for such misclassifications. The objective function and constraints of the underlying problem can be formulated as:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^l \xi_i \\ \text{s.t.} \quad & y_i (\mathbf{w}^T \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \xi_i \geq 0, \quad i = 1, 2, \dots, l \end{aligned} \quad (1)$$

where  $l$  is the number of training patterns,  $C$  is a parameter which gives a tradeoff between maximum margin and classification error, and  $y_i$ , being +1 or -1, is the target label of pattern  $\mathbf{x}_i$ . In the above

setup,  $\phi: X \rightarrow F$  is a mapping from the input space to the feature space  $F$  where patterns are separated, and  $\mathbf{w}^T \phi(\mathbf{x}_i) + b = 0$  is the hyperplane to be derived, with  $\mathbf{w}$  and  $b$  being weight vector and offset, respectively.

Eq. (1) can be solved by introducing the Lagrangian as follows:

$$L(\mathbf{w}, \xi, b, \alpha, \beta) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^l \xi_i - \sum_{i=1}^l \beta_i \xi_i - \sum_{i=1}^l \alpha_i [y_i (\mathbf{w}^T \phi(\mathbf{x}_i) + b) - 1 + \xi_i] \quad (2)$$

where  $\alpha_i \geq 0$  and  $\beta_i \geq 0$ ,  $i = 1, \dots, l$  are the Lagrange multipliers. Note that  $L$  has to be minimized with respect to  $\mathbf{w}$ ,  $b$  and  $\xi$  given  $\alpha$  and  $\beta$ , and then maximized with respect to  $\alpha$  and  $\beta$  given  $\mathbf{w}$ ,  $b$  and  $\xi$ . By setting partial derivatives of  $L$  to zero and substituting the result into Eq. (2), the variables  $\mathbf{w}$ ,  $b$ ,  $\xi$ , and  $\beta$  can be eliminated and Eq. (2) can be transformed into the following Wolfe dual form:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^l \alpha_i - \frac{1}{2} \sum_{i=1}^l \sum_{j=1}^l y_i y_j \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j) \\ \text{s.t.} \quad & C \geq \alpha_i \geq 0, i = 1, 2, \dots, l, \\ & \sum_{i=1}^l y_i \alpha_i = 0. \end{aligned} \quad (3)$$

Note that  $k(\mathbf{x}_i, \mathbf{x}_j)$  is a kernel function which represents the inner product of  $\phi(\mathbf{x}_i)$  and  $\phi(\mathbf{x}_j)$ , i.e.,  $k(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ . Eq. (3) can be rewritten in matrix form as

$$\begin{aligned} \max_{\alpha} \quad & \alpha^T \mathbf{e} - \frac{1}{2} \alpha^T G(K) \alpha \\ \text{s.t.} \quad & \alpha \geq 0, \\ & C \mathbf{e} - \alpha \geq 0, \\ & \alpha^T \mathbf{y} = 0 \end{aligned} \quad (4)$$

where  $\mathbf{e} = [1, 1, \dots, 1]^T$  and  $G(K) = \text{diag}(\mathbf{y}) K \text{diag}(\mathbf{y})$ . Note that  $K \in R^{l \times l}$  is the kernel matrix with  $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ ,  $i = 1, 2, \dots, l$  and  $j = 1, 2, \dots, l$ .

## 3. Multiple-kernel learning

Early SVM-based methods used a single-kernel function as shown in Eq. (4). This may encounter difficulties. The performance of a single-kernel method depends heavily on the choice of the kernel. Typically, a parameterized family of kernels, e.g., the Gaussian or polynomial kernel family, is chosen and the associated kernel parameters are tuned via cross-validation. If a dataset has varying local distributions, a single kernel may not be adequate to cope with such varying distributions. Usually, the decision boundary obtained is closer to the examples of a specific class, and may lead to a poor generalization ability. Kernel fusion has been proposed to deal with this problem. For example, a simple direct sum defined as  $k(\mathbf{x}_i, \mathbf{x}_j) \equiv \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle$ , where  $\Phi(\mathbf{x}) = [\phi_1(\mathbf{x}) \dots \phi_M(\mathbf{x})]^T$ , can be used to handle varying pattern distributions, and the corresponding kernel matrix can be easily written as  $K = K_1 + \dots + K_M$  where  $K_i$  is the kernel matrix constructed from  $\phi_i$ ,  $1 \leq i \leq M$ . This simple fusion can be generalized to a weighted combination of multiple kernels as follows:

$$K = \sum_{s=1}^M \mu_s K_s \quad (5)$$

where  $\mu_s$ ,  $1 \leq s \leq M$ , are weights. Obviously, we'd like  $\mu_s$  to be optimal. Multiple-kernel learning has been developed to derive optimal weights for Eq. (5). This approach allows the user to combine possibly heterogeneous data sources, and permits reducing heterogeneous data types to the common framework of kernel matrices and selecting the coefficients that favor those sources most useful in the classification decision. Note that several kernel functions

can be combined to provide the inner product between the mapped vectors in an appropriate feature space.

Lanckriet et al. (2004) proposed a multiple-kernel learning with a transduction setting. The generalized performance measure is minimized with respect to the combined kernel matrix  $\underline{K}$ , where  $\underline{K} = \sum_{s=1}^M \mu_s K_s \in \mathbb{R}^{l \times l}$  is obtained from both the training and testing patterns, and  $l$  is the total number of patterns. The objective function and constraints of the underlying problem are formulated as

$$\begin{aligned} \min_{\underline{K}} \quad & \max_{\alpha} \alpha^T \mathbf{e} - \frac{1}{2} \alpha^T G(K) \alpha \\ \text{s.t.} \quad & \alpha \geq 0, \\ & \mathbf{C}\mathbf{e} - \alpha \geq 0, \\ & \alpha^T \mathbf{y} = 0, \\ & \underline{K} \succeq 0, \\ & \text{trace}(\underline{K}) = c \end{aligned} \quad (6)$$

where  $K = \sum_{s=1}^M \mu_s K_s \in \mathbb{R}^{l \times l}$  obtained from the training patterns is the principal submatrix of  $\underline{K}$ ,  $\underline{K} \succeq 0$  means that  $\underline{K}$  is a positive semi-definite matrix, and  $c$  is a user-defined parameter. Eq. (6) is then transformed into the following SDP standard form:

$$\begin{aligned} \min_{t, \underline{K}, \mathbf{v}, \delta} \quad & t \\ \text{s.t.} \quad & \underline{K} \succeq 0, \\ & \text{trace}(\underline{K}) = c, \\ & \begin{bmatrix} G(K) & (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y}) \\ (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y})^T & t - 2C\delta^T \mathbf{e} \end{bmatrix} \succeq 0, \\ & \mathbf{v} \geq 0, \\ & \delta \geq 0 \end{aligned} \quad (7)$$

where  $t \in \mathbb{R}$ ,  $\mathbf{v} \in \mathbb{R}^l$ ,  $\delta \in \mathbb{R}^l$ , and  $\lambda \in \mathbb{R}$ . By converting Eq. (7) to the dual form, the optimal weights of Eq. (5) can be obtained by applying the primal–dual interior point method (Nesterov & Nemirovsky, 1994; Todd, Toh, & Tütüncü, 1998; Vandenberghe & Boyd, 1996).

#### 4. Proposed method

The original SDP method proposed by Lanckriet et al. (2004) has some disadvantages. The fourth constraint of Eq. (6) provides a better generalization ability on testing data, but it needs to check whether the combined matrix is positive semidefinite in each iteration. The last constraint of Eq. (6) controls overfitting, but allows the weights  $\mu_s$ ,  $s = 1, \dots, M$ , to be negative. As a result, the feasible region for the weights becomes huge and it is hard to search for optimal solutions. Besides,  $\underline{K}$  involves both the training and testing data. Therefore, the amount of time and space required is demanding.

Lanckriet et al. noted the issues. They discussed induction setting and non-negativity as a way of simplifying the model development process (Lanckriet et al., 2004). Based on their comments, We reformulate the SDP problem and incorporate two strategies to reduce the search complexity of the learning process. In the following, we first convert the multiple-kernel optimization problem to the primal SDP form. Then we convert the primal form to the dual SDP form. The problem can then be solved by the primal–dual interior point method (Nesterov & Nemirovsky, 1994; Todd et al., 1998; Vandenberghe & Boyd, 1996).

##### 4.1. Primal form

An induction setting is employed, with which  $\underline{K}$  is substituted by  $K$ . In addition, two strategies are incorporated to help narrow down the feasible region for the kernel weights. One strategy requires the kernel weights to be non-negative, and the other

restricts the sum of the weights to be one. Therefore, the multiple-kernel optimization problem for SVMs becomes:

$$\begin{aligned} \min_{\mu} \quad & \max_{\alpha} \alpha^T \mathbf{e} - \frac{1}{2} \alpha^T G(K) \alpha \\ \text{s.t.} \quad & \alpha \geq 0, \\ & \mathbf{C}\mathbf{e} - \alpha \geq 0, \\ & \alpha^T \mathbf{y} = 0, \\ & \mu \geq 0, \\ & \mathbf{e}^T \mu = 1 \end{aligned} \quad (8)$$

where  $K = \sum_{s=1}^M \mu_s K_s \in \mathbb{R}^{l \times l}$  is obtained from the training patterns and  $\mu \in \mathbb{R}^M$  is the weight vector of the kernel matrices. Now we introduce an auxiliary variable  $t \in \mathbb{R}$  that serves as an upper bound on the objective function, and Eq. (8) becomes:

$$\begin{aligned} \min_{t, \mu} \quad & t \\ \text{s.t.} \quad & t \geq \max_{\alpha \geq 0, \mathbf{C}\mathbf{e} - \alpha \geq 0, \alpha^T \mathbf{y} = 0} \alpha^T \mathbf{e} - \frac{1}{2} \alpha^T G(K) \alpha, \\ & \mu \geq 0, \\ & \mathbf{e}^T \mu = 1. \end{aligned} \quad (9)$$

The maximum on the right hand side of the first constraint in Eq. (9) can be found with the following Lagrangian:

$$L(\alpha, \mathbf{v}, \delta, \lambda) = \alpha^T \mathbf{e} - \frac{1}{2} \alpha^T G(K) \alpha + \mathbf{v}^T \alpha + \delta^T (\mathbf{C}\mathbf{e} - \alpha) + \lambda (\alpha^T \mathbf{y}) \quad (10)$$

where  $\mathbf{v} \in \mathbb{R}^l \geq 0$ ,  $\delta \in \mathbb{R}^l \geq 0$ , and  $\lambda \in \mathbb{R}$  are the Lagrange multipliers.  $L$  has to be maximized with respect to  $\alpha$  given  $\mathbf{v}$ ,  $\delta$ , and  $\lambda$ , and then minimized with respect to  $\mathbf{v}$ ,  $\delta$ , and  $\lambda$  given  $\alpha$ .

Setting partial derivatives of  $L$  with respect to  $\alpha$  equal to 0 yields the following:

$$\begin{aligned} \frac{\partial L}{\partial \alpha} &= \mathbf{e} - G(K) \alpha + \mathbf{v} - \delta + \lambda \mathbf{y} = 0 \\ \Rightarrow G(K) \alpha &= \mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y} \\ \Rightarrow \alpha &= (G(K))^{-1} (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y}). \end{aligned} \quad (11)$$

Substituting Eq. (11) into Eq. (10),  $\alpha$  can be eliminated and Eq. (10) is turned into the following dual form:

$$L = (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y})^T (G(K))^{-1} (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y}) + 2C\delta^T \mathbf{e}. \quad (12)$$

This implies that for any  $t > 0$ , the inequality constraint  $t \geq L$  holds if and only if there exist  $\mathbf{v} \geq 0$ ,  $\delta \geq 0$ , and  $\lambda$  such that

$$t \geq (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y})^T (G(K))^{-1} (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y}) + 2C\delta^T \mathbf{e}. \quad (13)$$

If  $K$  is positive semidefinite, it can be shown that  $G(K) = \text{diag}(\mathbf{y}) K \text{diag}(\mathbf{y})$  is also positive semidefinite as follows.

**Proposition 1.** If  $K$  is positive semidefinite,  $G(K) = \text{diag}(\mathbf{y}) K \text{diag}(\mathbf{y})$  is also positive semidefinite.

**Proof.** Let  $\mathbf{v}$  be any vector and  $\text{diag}(\mathbf{y}) = Y$ . Since  $Y^T = Y$  and  $Y^T Y$  is an identity matrix, we have  $\langle \mathbf{v}, YKY\mathbf{v} \rangle = \langle Y^T \mathbf{v}, KY\mathbf{v} \rangle = \langle Y\mathbf{v}, KY\mathbf{v} \rangle$ . Applying  $K \succeq 0$  to  $Y\mathbf{v}$ , we have

$$\begin{aligned} \langle Y\mathbf{v}, KY\mathbf{v} \rangle &\geq 0 \\ \Rightarrow \langle \mathbf{v}, YKY\mathbf{v} \rangle &\geq 0 \\ \Rightarrow YKY &\succeq 0 \\ \Rightarrow G(K) &\succeq 0. \end{aligned}$$

This completes the proof.  $\square$

Since  $G(K) \succeq 0$  and  $t - L \geq 0$ , by the Schur complement lemma (Abadir & Magnus, 2005; Vandenberghe & Boyd, 1996), we have:

$$P = \begin{bmatrix} G(K) & (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y}) \\ (\mathbf{e} + \mathbf{v} - \delta + \lambda \mathbf{y})^T & t - 2C\delta^T \mathbf{e} \end{bmatrix} \succeq 0. \quad (14)$$

Taking Eq. (14) into account, Eq. (9) can be expressed in the primal SDP form as follows:

$$\begin{aligned} \min_{t, \mu, \lambda, \mathbf{v}, \delta} \quad & t \\ \text{s.t.} \quad & P \succeq 0, \mathbf{v} \geq 0, \delta \geq 0, \mu \geq 0, \mathbf{e}^T \mu = 1 \end{aligned} \quad (15)$$

where  $t \in R$ ,  $\mu \in R^M$ ,  $\mathbf{v} \in R^l$ ,  $\delta \in R^l$ , and  $\lambda \in R$  are primal variables. Note that there are several kinds of constraints in Eq. (15). The first constraint is a linear matrix inequality (LMI) constraint, the next three are affine inequalities, and the last one is an affine equality constraint.

#### 4.2. Dual form

Next, we convert Eq. (15) to the dual form. Matrix  $P$  of Eq. (14) can be decomposed as

$$P = F_0 + tF_t + \lambda F_\lambda + \sum_{i=1}^M \mu_i F_{\mu_i} + \sum_{j=1}^l v_j F_{v_j} + \sum_{k=1}^l \delta_k F_{\delta_k} \quad (16)$$

where  $F_0, F_t, F_\lambda, F_{\mu_i}, F_{v_j}$ , and  $F_{\delta_k}$  are  $(l+1) \times (l+1)$  symmetric matrices, and

$$F_0 = \begin{bmatrix} \mathbf{O}_l & \mathbf{e} \\ \mathbf{e}^T & 0 \end{bmatrix}, \quad (17)$$

$$F_t = \begin{bmatrix} \mathbf{O}_l & \mathbf{O}_l \\ \mathbf{O}_l^T & 1 \end{bmatrix}, \quad (18)$$

$$F_\lambda = \begin{bmatrix} \mathbf{O}_l & \mathbf{y} \\ \mathbf{y}^T & 0 \end{bmatrix}, \quad (19)$$

$$F_{\mu_i} = \begin{bmatrix} G(\mu_i K_i) & \mathbf{O}_l \\ \mathbf{O}_l^T & 0 \end{bmatrix}, i = 1, \dots, M, \quad (20)$$

$$F_{v_j} = \begin{bmatrix} & & \mathbf{O}_{j-1} \\ & \mathbf{O}_l & 1 \\ & & \mathbf{O}_{l-j} \\ \mathbf{O}_{j-1}^T & 1 & \mathbf{O}_{l-j}^T & 0 \end{bmatrix}, j = 1, \dots, l, \quad (21)$$

$$F_{\delta_k} = \begin{bmatrix} & & \mathbf{O}_{k-1} \\ & \mathbf{O}_l & -1 \\ & & \mathbf{O}_{l-k} \\ \mathbf{O}_{k-1}^T & -1 & \mathbf{O}_{l-k}^T & -2C \end{bmatrix}, k = 1, \dots, l. \quad (22)$$

Note that  $\mathbf{O}_{mn}$  is an  $m \times n$  matrix with all elements being zero,  $\mathbf{O}_m$  is an  $m \times 1$  matrix with all elements being zero, and  $\mathbf{e}$  is an  $l \times 1$  matrix with all elements being one. Then Eq. (15) can be rewritten as follows:

$$\begin{aligned} \min_{t, \mu, \lambda, \mathbf{v}, \delta} \quad & t \\ \text{s.t.} \quad & F_0 + tF_t + \lambda F_\lambda + \sum_{i=1}^M \mu_i F_{\mu_i} + \sum_{j=1}^l v_j F_{v_j} + \sum_{k=1}^l \delta_k F_{\delta_k} \succeq 0, \\ & \mu_i \geq 0, i = 1, 2, \dots, M, \\ & v_j \geq 0, j = 1, 2, \dots, l, \\ & \delta_k \geq 0, k = 1, 2, \dots, l, \\ & \sum_{i=1}^M \mu_i - 1 = 0. \end{aligned} \quad (23)$$

By applying the Lagrangian, we have

$$\begin{aligned} L(t, \mu, \lambda, \mathbf{v}, \delta) = & t \\ & - \left( F_0 + tF_t + \lambda F_\lambda + \sum_{i=1}^M \mu_i F_{\mu_i} + \sum_{j=1}^l v_j F_{v_j} + \sum_{k=1}^l \delta_k F_{\delta_k} \right) \\ & \cdot Z - \sum_{i=1}^M p_i \mu_i - \sum_{j=1}^l q_j v_j - \sum_{k=1}^l r_k \delta_k \\ & + s \left( \sum_{i=1}^M \mu_i - 1 \right) \\ = & -F_0 \cdot Z - s - t(F_t \cdot Z - 1) - \lambda(F_\lambda \cdot Z) \\ & - \sum_{i=1}^M \mu_i (F_{\mu_i} \cdot Z + p_i - s) - \sum_{j=1}^l v_j (F_{v_j} \cdot Z + q_j) \\ & - \sum_{k=1}^l \delta_k (F_{\delta_k} \cdot Z + r_k) \end{aligned} \quad (24)$$

where  $Z = \mathbf{z}\mathbf{z}^T \succeq 0$  and  $\mathbf{z}$  is an  $(l+1) \times 1$  matrix. Note that  $z_v, v = 1, \dots, l+1, p_i \geq 0, i = 1, \dots, M, q_j \geq 0, j = 1, \dots, l, r_k \geq 0, k = 1, \dots, l$ , and  $s \in R$  are Lagrange multipliers. Setting partial derivatives of  $L$  equal to 0 yields the following:

$$\begin{aligned} \frac{\partial L}{\partial t} &= F_t \cdot Z - 1 = 0, \\ \frac{\partial L}{\partial \lambda} &= F_\lambda \cdot Z = 0, \\ \frac{\partial L}{\partial \mu_i} &= F_{\mu_i} \cdot Z + p_i - s = 0, i = 1, \dots, M, \\ \frac{\partial L}{\partial v_j} &= F_{v_j} \cdot Z + q_j = 0, j = 1, \dots, l, \\ \frac{\partial L}{\partial \delta_k} &= F_{\delta_k} \cdot Z + r_k = 0, k = 1, \dots, l. \end{aligned} \quad (25)$$

Substituting Eq. (25) into Eq. (24), we have the following Wolfe dual form:

$$\begin{aligned} \max \quad & -F_0 \cdot Z - s \\ \text{s.t.} \quad & F_t \cdot Z = 1, \\ & F_\lambda \cdot Z = 0, \\ & F_{\mu_i} \cdot Z + p_i - s = 0, p_i \geq 0, i = 1, \dots, M, \\ & F_{v_j} \cdot Z + q_j = 0, q_j \geq 0, j = 1, \dots, l, \\ & F_{\delta_k} \cdot Z + r_k = 0, r_k \geq 0, k = 1, \dots, l, \\ & Z \succeq 0. \end{aligned} \quad (26)$$

Note that the last constraint of Eq. (26) is an LMI constraint, and the others are affine equalities or inequalities.

Now, by applying the primal–dual interior point method, together with the following KKT complementary conditions:

$$\begin{aligned} 0 &= \left( F_0 + tF_t + \lambda F_\lambda + \sum_{i=1}^M \mu_i F_{\mu_i} + \sum_{j=1}^l v_j F_{v_j} + \sum_{k=1}^l \delta_k F_{\delta_k} \right) \cdot Z, \\ 0 &= p_i \mu_i, i = 1, \dots, M, \\ 0 &= q_j v_j, j = 1, \dots, l, \\ 0 &= r_k \delta_k, k = 1, \dots, l, \end{aligned} \quad (27)$$

we can solve the dual variables  $\mathbf{z}, \mathbf{p}, \mathbf{q}, \mathbf{r}$ , and  $s$ , and subsequently, the primal variables  $t, \mu, \mathbf{v}, \delta$ , and  $\lambda$ . Then the Lagrange multipliers ( $\alpha$ ) of the multiple-kernel SVM, Eq. (8), can be solved by Eq. (11). Note that  $G(K)$  may not be always invertible. We may overcome this difficulty by using the pseudoinverse  $(G(K))^+$  instead of  $(G(K))^{-1}$  (Tsang & Kwok, 2006) in Eq. (11).



### 4.3. Complexity analysis

In solving the SDP problem, clearly it takes a lot more efforts to check LMI constraints than to check affine equalities or inequalities. For simplicity, we ignore affine equalities and inequalities, and consider a simple case of SDP with multiple LMI constraints as follows:

$$\begin{aligned} \min_{\mathbf{u}} \quad & \mathbf{c}^T \mathbf{u} \\ \text{s.t.} \quad & F^j(\mathbf{u}) = F_0^j + u_1 F_1^j + \dots + u_n F_n^j \succeq 0, j = 1, \dots, L \end{aligned} \quad (28)$$

where  $\mathbf{u} \in \mathbb{R}^n$  is the vector of the primal variables,  $\mathbf{c} \in \mathbb{R}^n$  is the objective vector,  $F_i^j \in \mathbb{R}^{n_j \times n_j}$ ,  $i = 0, 1, \dots, n$  and  $j = 1, \dots, L$ , are given symmetric matrices, and  $L$  is the number of LMI constraints. The worst-case time complexity of Eq. (28) for each iteration is  $O(n^2 \sum_{j=1}^L n_j^2)$ . Solving Eq. (28) by the primal–dual interior point method needs  $O\left(\left(\sum_{j=1}^L n_j\right)^{0.5}\right)$  iterations (Vandenberghe & Boyd, 1996). Hence, the total worst-case time complexity is  $O\left(n^2 \left(\sum_{j=1}^L n_j^2\right) \left(\sum_{j=1}^L n_j\right)^{0.5}\right)$ .

Let us compare the total worst-case time complexity of the original SDP formulation (Lanckriet et al., 2004) with that of our proposed SDP formulation. From Eqs. (7) and (28), we have  $n = M + l + 2 \approx M + 2l$ ,  $L = 2$ ,  $n_1 = l$ , and  $n_2 = l + 1 \approx l$ . The total worst-case time complexity of the original SDP method (Lanckriet et al., 2004) is  $O((M + 2l)^2(l^2 + l^2)(l + l)^{0.5})$ . From Eq. (15) and Eq. (28), we have  $n = M + l + 2 \approx M + 2l$ ,  $L = 1$ , and  $n_1 = l + 1 \approx l$ . The total worst-case time complexity of our proposed formulation is  $O((M + 2l)^2(l^2)(l)^{0.5})$ . Also, the two adopted strategies, for avoiding checking positive semidefiniteness of the kernel matrix and restricting the sum of the kernel weights equal to 1, respectively, help narrow down the feasible region for the kernel weights. Obviously, the total worst-case time complexity of our proposed method is smaller than that of the original SDP method (Lanckriet et al., 2004).

## 5. Experimental results

In this section, we show experimental results to demonstrate the effectiveness of our proposed method. For convenience, we abbreviate the multiple-kernel SVM with our SDP learning as SDPI, and the multiple-kernel SVM with the original SDP (Lanckriet et al., 2004) as SDPT. In the first experiment, we compare the performance of the single kernel SVM and SDPI on a simple set of synthetic data. In Experiment II, we compare the performance of SDPI and SDPT on six synthetic data sets. In Experiment III, we compare the single kernel SVM, SDPI, and SDPT on four real data sets taken from the UCI Repository (Newman, avd, Blake, & Merz, 1998) and the Statlog collection (Michie, Spiegelhalter, & Taylor, 1994), respectively. For each experiment, we use SeDuMi (Sturm, 1999) as the SDP solver, and the standard form for SeDuMi is prepared by the application of YALMIP (Löfberg, 2004).

### 5.1. Experiment I

For this experiment, we consider 100 two-dimensional patterns of two classes, class +1 and class −1, as shown in Fig. 1. Among them, 50 belong to class +1 and the remaining belong to class −1. Each pattern is in the form of  $(5 \cos(\theta), 5 \sin(\theta))$  where  $\theta$  is randomly generated. Also, a Gaussian noise of different types is added to class +1 and class −1 patterns, respectively. In the figure, a pattern of class +1 is marked with '+' and a pattern of class −1 is marked with 'x'. Note that class −1 has a dense distribution and class +1 has a sparse distribution. The decision boundaries

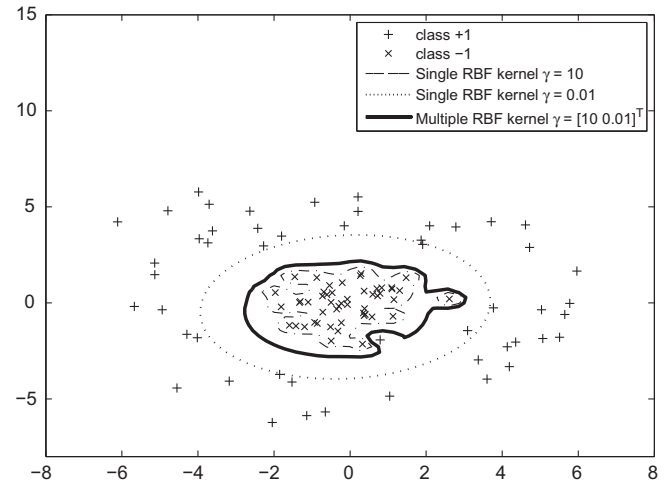


Fig. 1. Decision boundaries obtained in Experiment I.

for separating the two classes obtained by the single radial basis function (RBF) kernel SVM with different values of  $\gamma$  (width), i.e.,  $\gamma \in \{0.01, 10\}$ , are shown in Fig. 1. In this figure, both  $x$  and  $y$  coordinates indicate real numbers. Note that a RBF kernel with width  $\gamma$  is expressed as

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2).$$

Obviously, the decision boundary for  $\gamma = 10$  is too specific to the data patterns of class −1. On the other hand, the decision boundary for  $\gamma = 0.01$  is too specific to the data patterns of class +1. Neither of these boundaries is good and can have good generalization ability.

Next, we apply SDPI to the dataset. A two-kernel learning is adopted for this case, i.e.,  $K = \mu_1 \times K_1 + \mu_2 \times K_2$ , where  $K_1$  and  $K_2$  are RBFs with  $\gamma_1 = 10$  and  $\gamma_2 = 0.01$ , respectively. The optimal weights  $\mu_1$  and  $\mu_2$  obtained by our SDPI are 0.726 and 0.274, respectively. As shown in Fig. 1, the decision boundary obtained is close to the middle of the separating zone and hence can increase the generalization ability.

### 5.2. Experiment II

In this experiment, we compare the performance of SDPI and SDPT on six synthetic datasets. The same two-kernel learning as previously used in Experiment I is also adopted for SDPI and SDPT. That is,  $K = \mu_1 \times K_1 + \mu_2 \times K_2$ , where  $K_1$  and  $K_2$  are RBFs with  $\gamma_1 = 10$  and  $\gamma_2 = 0.01$ , respectively. The six datasets were generated by using the libraries and toolboxes provided in The MathWorks, Inc. (1998), and are denoted as DS-I to DS-VI. Each dataset contains 100 training patterns and 50 testing patterns. For DS-I, both the training and testing data are generated in the same way as in Experiment I. This dataset is linearly separable in the feature space. The decision boundaries obtained by SDPI and SDPT are similar to each other. However, SDPI runs substantially faster than SDPT. SDPI spent 3.078 s on training, while SDPT spent 7.672 s.

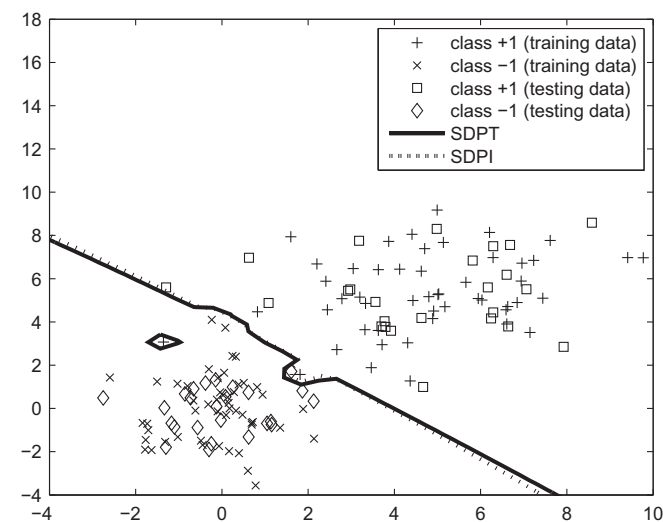
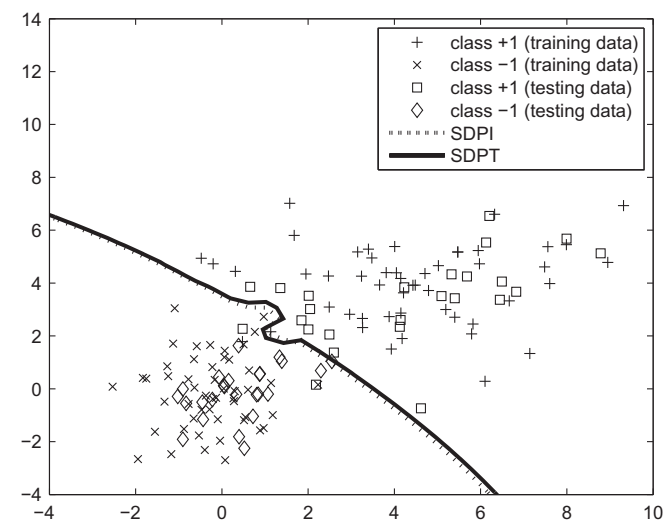
In the second dataset, DS-II, positive patterns and negative patterns are generated with different mean vectors, and are required to be linearly separable with margin equal to 1 in the input space. Again, the decision boundaries obtained by SDPI and SDPT are similar to each other, and SDPI runs substantially faster than SDPT. SDPI spent 2.703 s on training, while SDPT spent 7.171 s.

The data patterns for the datasets, DS-III to DS-VI, are drawn from the 2-variate normal distribution with different mean vectors and covariance matrices, as specified in Table 1. Note that DS-VI has more overlapped patterns than the other datasets, and is the

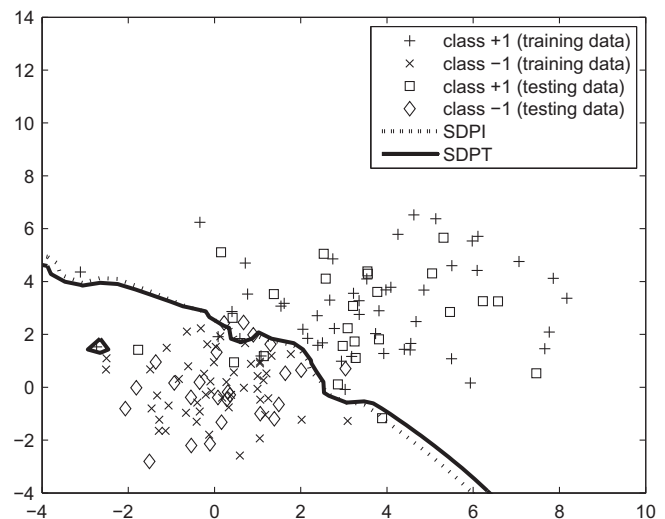
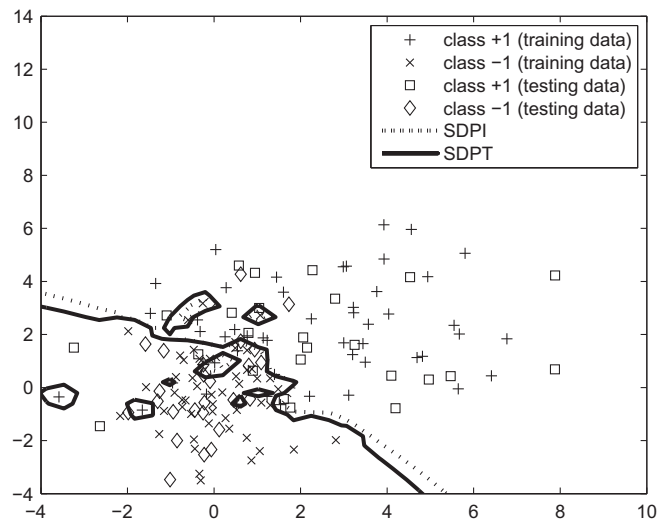
**Table 1**

The four datasets, DS-III–DS-VI, used in Experiment II.

| Class | Parameters        | Datasets                                           |                                                    |
|-------|-------------------|----------------------------------------------------|----------------------------------------------------|
|       |                   | DS-III                                             | DS-IV                                              |
| 1     | Mean vector       | $[00]^T$                                           | $[00]^T$                                           |
|       | Covariance matrix | $\begin{bmatrix} 1 & 0.1 \\ 0.1 & 2 \end{bmatrix}$ | $\begin{bmatrix} 1 & 0.1 \\ 0.1 & 2 \end{bmatrix}$ |
| -1    | Mean vector       | $[55]^T$                                           | $[44]^T$                                           |
|       | Covariance matrix | $\begin{bmatrix} 5 & 0.5 \\ 0.5 & 3 \end{bmatrix}$ | $\begin{bmatrix} 5 & 0.5 \\ 0.5 & 3 \end{bmatrix}$ |
|       |                   | DS-V                                               | DS-VI                                              |
|       |                   |                                                    |                                                    |
| 1     | Mean vector       | $[00]^T$                                           | $[00]^T$                                           |
|       | Covariance matrix | $\begin{bmatrix} 1 & 0.1 \\ 0.1 & 2 \end{bmatrix}$ | $\begin{bmatrix} 1 & 0.1 \\ 0.1 & 2 \end{bmatrix}$ |
| -1    | Mean vector       | $[33]^T$                                           | $[22]^T$                                           |
|       | Covariance matrix | $\begin{bmatrix} 5 & 0.5 \\ 0.5 & 3 \end{bmatrix}$ | $\begin{bmatrix} 5 & 0.5 \\ 0.5 & 3 \end{bmatrix}$ |

**Fig. 2.** Decision boundaries obtained for DS-III in Experiment II.**Fig. 3.** Decision boundaries obtained for DS-IV in Experiment II.

most difficult one to be separated into two classes. The decision boundaries obtained by SDPI and SDPT on these four synthetic datasets are shown in Figs. 2–5, respectively. In these figures, a testing pattern of class +1 is marked with '□' and a testing pattern

**Fig. 4.** Decision boundaries obtained for DS-V in Experiment II.**Fig. 5.** Decision boundaries obtained for DS-VI in Experiment II.

of class -1 is marked with '◇'. Obviously, SDPI can obtain decision boundaries similar to those obtained by SDPT. However, SDPT is more likely to cause overfitting. For example, there is an outlier in DS-III and DS-V. SDPT gets annoyed by the outlier, but SDPI does not, as shown in Figs. 2 and 4, respectively. Therefore, SDPI can have a better generalization ability than SDPT. Table 2 shows performance comparisons between SDPI and SDPT on the four datasets. We can see that SDPI works almost as accurately as SDPT. For the DS-VI dataset, SDPI even gets a significantly higher testing accuracy rate than SDPT, i.e., 86% vs. 78%. Note that the sum of the kernel weights is unity for SDPI for each case. Also, note that SDPI runs much faster than SDPT.

### 5.3. Experiment III

In this experiment, we compare the performance of the single kernel SVM, SDPI, and SDPT, on some real datasets. Three datasets, 'breast cancer', 'ionosphere', and 'sonar', are taken from the UCI Repository (Newman et al., 1998) and one dataset, 'heart', is taken from the Statlog collection (Michie et al., 1994). A brief description

**Table 2**

Performance comparison on DS-III–DS-VI in Experiment II.

| Method |                       | Datasets    |             |
|--------|-----------------------|-------------|-------------|
|        |                       | DS-III      | DS-IV       |
| SDPT   | Training accuracy (%) | 100.000     | 99.000      |
|        | Testing accuracy (%)  | 98.000      | 96.000      |
|        | Time (s)              | 7.688       | 7.891       |
|        | $\mu_1/\mu_2$         | 1.256/0.744 | 1.130/0.870 |
|        |                       |             |             |
| SDPI   | Training accuracy (%) | 99.000      | 99.000      |
|        | Testing accuracy (%)  | 98.000      | 94.000      |
|        | Time (s)              | 2.657       | 2.937       |
|        | $\mu_1/\mu_2$         | 0.478/0.522 | 0.437/0.563 |
|        |                       |             |             |
|        |                       | DS-V        | DS-VI       |
| SDPT   | Training accuracy (%) | 98.000      | 98.000      |
|        | Testing accuracy (%)  | 84.000      | 78.000      |
|        | Time (s)              | 8.093       | 7.344       |
|        | $\mu_1/\mu_2$         | 1.361/0.639 | 1.743/0.257 |
|        |                       |             |             |
| SDPI   | Training accuracy (%) | 97.000      | 95.000      |
|        | Testing accuracy (%)  | 86.000      | 86.000      |
|        | Time (s)              | 2.578       | 2.766       |
|        | $\mu_1/\mu_2$         | 0.584/0.416 | 0.738/0.212 |
|        |                       |             |             |

**Table 3**

The four real datasets used in Experiment III.

| Datasets      | No. of patterns | No. of dimensions |
|---------------|-----------------|-------------------|
| Breast cancer | 683             | 10                |
| Ionosphere    | 351             | 33                |
| Heart         | 270             | 13                |
| Sonar         | 208             | 60                |

**Table 4**

Accuracy rates of the single kernel SVM, with three different kernel types, in Experiment III.

| Kernel             | Datasets      |            |
|--------------------|---------------|------------|
|                    | Breast cancer | Ionosphere |
| Linear             | 91.232        | 79.206     |
| Polynomial $d = 2$ | 89.033        | 84.905     |
| RBF $\gamma = 1$   | 94.463        | 92.318     |
|                    | Heart         | Sonar      |
| Linear             | 83.333        | 53.786     |
| Polynomial $d = 2$ | 82.222        | 53.786     |
| RBF $\gamma = 1$   | 75.926        | 85.571     |

of these datasets is given in Table 3. For the sake of easy comparison, we scale the values of all the training patterns within the range of  $[-1, 1]$ . Also, a 10-fold cross validation is used to evaluate the performance of each method. A dataset is randomly divided into 10 equal parts. Each time nine parts are used for training and the remaining part is used for testing. Therefore, 10 runs are done for each dataset and the result is the average of the 10 runs.

Table 4 shows the results obtained by the single kernel SVM with three different types of kernels, i.e., linear, polynomial with  $d = 2$ , and RBF with  $\gamma = 1$ . Table 5 shows the results obtained by SDPT and SDPI, with the combination of the above three different kernels. From these tables, we can see that SDPI outperforms the single kernel SVM, as similarly indicated in Lanckriet et al. (2004). Table 5 shows that SDPI has higher accuracy rates than SDPT for the datasets 'ionosphere' and 'heart'. For the 'breast cancer' dataset, SDPI and SDPT have the same accuracy. However, SDPT has a higher accuracy rate than SDPI for the 'sonar' dataset. Table 6 shows the kernel weights obtained from SDPI and SDPT for each dataset. Note that for SDPT, negative weights are allowed. For instance, the weight with the linear kernel for the 'ionosphere' dataset is  $-0.524$ . For SDPT, a high kernel weight indicates that the

**Table 5**

Performance comparison of SDPI and SDPT, combining three different kernel types, in Experiment III.

| Method |              | Datasets      |            |
|--------|--------------|---------------|------------|
|        |              | Breast cancer | Ionosphere |
| SDPT   | Accuracy (%) | 96.643        | 95.159     |
|        | Time (s)     | 1669.400      | 158.342    |
|        |              |               |            |
| SDPI   | Accuracy (%) | 96.643        | 95.444     |
|        | Time (s)     | 763.569       | 75.374     |
|        |              |               |            |
|        |              | Heart         | Sonar      |
| SDPT   | Accuracy (%) | 82.963        | 87.429     |
|        | Time (s)     | 62.958        | 29.025     |
|        |              |               |            |
| SDPI   | Accuracy (%) | 83.333        | 85.571     |
|        | Time (s)     | 31.510        | 13.794     |
|        |              |               |            |

**Table 6**

Kernel weights obtained for three different kernel types in Experiment III.

| Method     | Breast cancer             |                    |                       |
|------------|---------------------------|--------------------|-----------------------|
|            | $\mu_{\text{polynomial}}$ | $\mu_{\text{RBF}}$ | $\mu_{\text{linear}}$ |
| SDPT       | 0.379                     | 2.346              | 0.275                 |
| SDPI       | 0.126                     | 0.782              | 0.092                 |
| Ionosphere |                           |                    |                       |
| SDPT       | 2.037                     | 1.487              | $-0.524$              |
| SDPI       | 0.475                     | 0.525              | 0.000                 |
| Heart      |                           |                    |                       |
| SDPT       | 0.000                     | 2.366              | 0.634                 |
| SDPI       | 0.000                     | 0.789              | 0.211                 |
| Sonar      |                           |                    |                       |
| SDPT       | $-2.085$                  | 3.503              | 1.582                 |
| SDPI       | 0.000                     | 1.000              | 0.000                 |

corresponding kernel matrix contributes more power to classification, while a negative kernel weight indicates that the corresponding kernel matrix provides complementary information for classification. For SDPI, a kernel with zero weight has no contribution to the construction of the decision boundary, while a high kernel weight indicates that the corresponding kernel matrix offers more contribution to the construction of the decision boundary. For instance, the weight with the linear kernel for the 'ionosphere' dataset is zero, indicating that the linear kernel is negligible. Regarding the time performance, SDPI runs substantially faster than SDPT.

Table 7 shows the results obtained by the single kernel SVM with five different RBF kernels, i.e.,  $\gamma \in \{50, 5, 0.5, 0.05, 0.005\}$ . Table 8 shows the results obtained by SDPT, and SDPI, with the combination of the above five different RBF kernels. From Table 8, we can see that

**Table 7**

Accuracy rates of the single RBF kernel SVM in Experiment III.

| RBF kernel       | Datasets      |            |
|------------------|---------------|------------|
|                  | Breast cancer | Ionosphere |
| $\gamma = 50$    | 84.671        | 64.692     |
| $\gamma = 5$     | 89.474        | 66.675     |
| $\gamma = 0.5$   | 96.201        | 92.595     |
| $\gamma = 0.05$  | 96.061        | 92.889     |
| $\gamma = 0.005$ | 95.325        | 89.175     |
|                  | Heart         | Sonar      |
| $\gamma = 50$    | 55.556        | 55.786     |
| $\gamma = 5$     | 64.074        | 83.143     |
| $\gamma = 0.5$   | 79.630        | 84.357     |
| $\gamma = 0.05$  | 82.593        | 84.143     |
| $\gamma = 0.005$ | 83.333        | 79.286     |

**Table 8**

Performance comparison of SDPI and SDPT, combining five different RBF kernels, in Experiment III.

| Method |              | Datasets      |            |
|--------|--------------|---------------|------------|
|        |              | Breast cancer | Ionosphere |
| SDPT   | Accuracy (%) | 97.080        | 94.310     |
|        | Time (s)     | 4798.200      | 183.116    |
| SDPI   | Accuracy (%) | 96.643        | 93.744     |
|        | Time (s)     | 890.322       | 85.392     |
|        |              | Heart         | Sonar      |
| SDPT   | Accuracy (%) | 82.590        | 88.071     |
|        | Time (s)     | 70.284        | 34.462     |
| SDPI   | Accuracy (%) | 83.333        | 87.214     |
|        | Time (s)     | 40.545        | 18.488     |

**Table 9**

Kernel weights obtained for five different RBF kernels in Experiment III.

| Method     | Breast cancer     |                  |                    |                     |                      |
|------------|-------------------|------------------|--------------------|---------------------|----------------------|
|            | $\mu_{\gamma=50}$ | $\mu_{\gamma=5}$ | $\mu_{\gamma=0.5}$ | $\mu_{\gamma=0.05}$ | $\mu_{\gamma=0.005}$ |
| SDPT       | 1.192             | 2.711            | 0.527              | 1.516               | −0.946               |
| SDPI       | 0.021             | 0.481            | 0.423              | 0.085               | 0.000                |
| Ionosphere |                   |                  |                    |                     |                      |
| SDPT       | 0.913             | 0.996            | 2.368              | 0.955               | −0.232               |
| SDPI       | 0.000             | 0.000            | 0.818              | 0.182               | 0.000                |
| Heart      |                   |                  |                    |                     |                      |
| SDPT       | 0.241             | 4.063            | −0.517             | 2.504               | −1.291               |
| SDPI       | 0.217             | 0.428            | 0.119              | 0.236               | 0.000                |
| Sonar      |                   |                  |                    |                     |                      |
| SDPT       | −0.646            | 4.099            | 2.590              | −3.336              | 2.292                |
| SDPI       | 0.000             | 0.711            | 0.289              | 0.000               | 0.000                |

SDPI has a higher accuracy rate than SDPT for the ‘heart’ dataset, but has lower accuracy rates than SDPT for the ‘breast cancer’, ‘ionosphere’ and ‘sonar’ datasets. This may be attributed to the narrower search region used in SDPI. Nevertheless, the accuracy difference is very small. Table 9 shows the kernel weights obtained from SDPI and SDPT for each dataset. Regarding the time performance, SDPI runs substantially faster, e.g., over five times, than SDPT. Also, SDPI has higher accuracy rates than all the single RBF kernel SVMs. Notice that, from Table 7, the optimal hyperparameters involved in the single kernel SVM are different for different datasets. For example, for the ‘breast cancer’ dataset, the best single RBF kernel SVM is the one with  $\gamma = 0.5$ , while for the ‘heart’ dataset, the best single RBF kernel SVM is the one with  $\gamma = 0.005$ . This problem can be alleviated with the application of SDPI or SDPT. All the kernels can contribute and the contributing weight of each kernel is obtained automatically. Therefore, SDPI and SDPT provide a feasible means to solve the issue of determining hyperparameter values in kernel-based methods. From Table 6 and 9, we can see that the number of the involved kernels in SDPI can be fewer than that in SDPT.

We compare SDPI and SDPT with larger hyperparameter sets. The results obtained from the combination of ten RBF kernels, i.e.,

$$\gamma \in \{100, 50, 10, 5, 1, 0.5, 0.1, 0.05, 0.01, 0.005\},$$

are shown in Table 10. Table 11 shows the results obtained from the combination of twenty different RBF kernels, i.e.,

$$\gamma \in \{500, 200, 100, 90, 80, 70, 60, 50, 40, 30, 20, 10, 5, 1, 0.5, 0.1, 0.05, 0.01, 0.005, 0.001\}.$$

Note that the entries marked with ‘−’ indicate that SDPT failed running through the ‘breast cancer’ dataset due to the occurrence of out-of-memory errors.

**Table 10**

Performance comparison of SDPI and SDPT, combining ten different RBF kernels, in Experiment III.

| Method |              | Datasets      |            |
|--------|--------------|---------------|------------|
|        |              | Breast cancer | Ionosphere |
| SDPT   | Accuracy (%) | 97.384        | 93.730     |
|        | Time (s)     | 3827.323      | 336.213    |
| SDPI   | Accuracy (%) | 97.080        | 94.020     |
|        | Time (s)     | 1117.000      | 127.837    |
|        |              | Heart         | Sonar      |
| SDPT   | Accuracy (%) | 70.741        | 84.429     |
|        | Time (s)     | 137.2077      | 65.627     |
| SDPI   | Accuracy (%) | 83.333        | 87.071     |
|        | Time (s)     | 52.361        | 24.787     |

**Table 11**

Performance comparison of SDPI and SDPT, combining twenty different RBF kernels, in Experiment III.

| Method |              | Datasets      |            |
|--------|--------------|---------------|------------|
|        |              | Breast cancer | Ionosphere |
| SDPT   | Accuracy (%) | –             | 71.024     |
|        | Time (s)     | –             | 161.894    |
| SDPI   | Accuracy (%) | 96.974        | 94.016     |
|        | Time (s)     | 1490.392      | 168.236    |
|        |              | Heart         | Sonar      |
| SDPT   | Accuracy (%) | 52.222        | 68.357     |
|        | Time (s)     | 80.925        | 35.197     |
| SDPI   | Accuracy (%) | 83.333        | 87.071     |
|        | Time (s)     | 71.607        | 31.408     |

In SDPT, it is required that  $\sum_i^M \mu_i = H$  where  $H$  is the size of the hyperparameter set or the number of involved kernels. Also,  $\mu_i$  can be positive or negative. When the size of the hyperparameter set,  $H$ , increases, the variation of  $\mu_i$  also increases. This means that the search space also increases and it is harder to get optimal solutions. Therefore, the testing accuracy obtained by SDPT may decrease as the size of the hyperparameter set increases. Take ‘sonar’ as an example. For five kernels, the testing accuracy obtained by SDPT is 88.071% as shown in Table 8. For ten kernels, the testing accuracy obtained by SDPT drops to 84.429% as shown in Table 10. For 20 kernels, the testing accuracy obtained by SDPT further drops to 68.357% as shown in Table 11. However, SDPI provides about 87% in testing accuracy in all these three cases. The model used in Lanckriet et al. (2004) is more robust since no non-negativity constraints are imposed on the kernel weights. However, it is harder to find optimal solutions due to the huge search space encountered, especially as the number of kernels increases. The non-negativity constraints narrow down the search space and optimal solutions can be found more easily.

## 6. Conclusions

We have reformulated the SDP problem for multiple-kernel learning with an induction setting. The kernel matrix involved is constructed from the training patterns only, thus the required amount of time and space is reduced. Two strategies, non-negativity and sum-to-one, are incorporated for helping narrow down the search space for kernel weights and greatly reducing the computation complexity required to solve the SDP problem. The induction setting and the non-negativity assumption were discussed in Lanckriet et al. (2004) as a way of simplifying the model development process. Experimental results have shown that our proposed method runs efficiently in multiple-kernel learning. Besides, SDPI was successfully applied in detecting counterfeit banknotes (Yeh,



Su, & Lee, 2011). Note that if the size of the training set is too large, both SDPT and SDPI may fail. In this case, the use of large-scale semidefinite programming may be required.

## References

- Abadir, K. M., & Magnus, J. R. (2005). *Matrix algebra*. Cambridge, UK: Cambridge University Press.
- Bach, F. R., Lanckriet, G. R. G., & Jordan, M. I. (2004). Multiple kernel learning, conic duality, and the SMO algorithm. In *Proceedings of the 21st international conference on machine learning* (pp. 6–13).
- Bennett, K. P., Momma, M., & Embrechts, M. J. (2002). MARK: A boosting algorithm for heterogeneous kernel models. In *Proceedings of the 8th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 24–31).
- Chapelle, O., Vapnik, V., Bousquet, O., & Mukherjee, S. (2002). Choosing multiple parameters for support vector machines. *Machine Learning*, 46(1–3), 131–159.
- Cortes, C., & Vapnik, V. (1995). Support-vector network. *Machine Learning*, 20(3), 273–297.
- Crammer, K., Keshet, J., & Singer, Y. (2003). Kernel design using boosting. In S. Becker, S. Thrun, & K. Obermayer (Eds.), *Advances in neural information processing systems* (Vol. 15, pp. 537–544). USA: MIT Press, Cambridge, MA.
- Cristianini, N., & Shawe-Taylor, J. (2000). *An introduction to support vector machines and other kernel-based learning methods*. Cambridge, UK: Cambridge University Press.
- Drucker, H., Wu, D., & Vapnik, V. (1999). Support vector machines for spam categorization. *IEEE Transactions on Neural Networks*, 10(5), 1048–1054.
- Duan, K., Keerthi, S., & Poo, A. N. (2003). Evaluation of simple performance measures for tuning SVM hyperparameters. *Neurocomputing*, 51(4), 41–59.
- Friedrichs, F., & Igel, C. (2005). Evolutionary tuning of multiple svm parameters. *Neurocomputing*, 64(Complete), 107–117.
- Golub, G. H., Heath, M., & Wahba, G. (1979). Generalized cross-validation as a method for choosing a good ridge parameter. *Technometrics*, 21, 215–223.
- Gonen, M., & Alpaydin, E. (2008). Localized multiple kernel learning. In *Proceedings of the 25th international conference on machine learning* (pp. 352–359).
- Guo, G., Li, S. Z., & Chan, K. (2000). Face recognition by support vector machines. In *Proceedings of the 4th IEEE international conference on automatic face and gesture recognition* (pp. 196–201).
- Hou, Y., Hsu, W., Lee, M. L., & Bystroff, C. (2003). Efficient remote homology detection using local structure. *Bioinformatics*, 19(17), 2294–2301.
- Hsu, C.-W., Chang, C.-C., Lin, & C.-J. (2003). *A practical guide to support vector classification*. Technical Report, National Taiwan University.
- Kecman, V. (2001). *Learning and soft computing, support vector machines, neural networks and fuzzy logic models*. Cambridge, MA, USA: The MIT Press.
- Khandoker, A. H., Palaniswami, M., & Karmakar, C. K. (2009). Support vector machines for automated recognition of obstructive sleep apnea syndrome from ECG recordings. *IEEE Transactions on Information Technology in Biomedicine*, 13(11), 37–48.
- Kim, K. I., Jung, K., Park, S. H., & Kim, H. J. (2002). Support vector machines for texture classification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(11), 1542–1550.
- Kwok, J. T.-Y. (2000). The evidence framework applied to support vector machines. *IEEE Transactions on Neural Networks*, 11(5), 1162–1173.
- Lanckriet, G. R. G., Cristianini, N., Bartlett, P., Ghaoui, L. E., & Jordan, M. I. (2004). Learning the kernel matrix with semidefinite programming. *Journal of Machine Learning Research*, 5, 27–72.
- Löfberg, J. (2004). YALMIP: A toolbox for modeling and optimization in MATLAB. In *Proceedings of the 13th IEEE international symposium on the computer aided control systems design* (pp. 284–289).
- Michie, D., Spiegelhalter, D. J., & Taylor, C. C. (1994). *Machine learning. Neural and statistical classification*. New York, USA: Ellis Horwood.
- Nesterov, Y. E., & Nemirovsky, A. S. (1994). *Interior point polynomial methods in convex programming: Theory and applications, society for industrial and applied mathematics. Siam studies in applied mathematics* (Vol. 13). Philadelphia, USA.
- Newman, D. J., Avd, S. H., Blake, C. L., & Merz, C. J. (1998). *UCI repository of machine learning databases*.
- Ong, C. S., Smola, A. J., & Williamson, R. C. (2005). Learning the kernel with hyperkernels. *Journal of Machine Learning Research*, 6, 1043–1071.
- Platt, J. C. (1999). Fast training of support vector machines using sequential minimal optimization. In B. Schölkopf, C. J. C. Burges, & A. J. Smola (Eds.), *Advances in kernel methods: support vector learning* (pp. 185–208). Cambridge, MA, USA: MIT Press.
- Pontil, M., & Verri, A. (1998). Support vector machines for 3D object recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(6), 637–646.
- Rakotomamonjy, A., Bach, F. R., Canu, S., & Grandvalet, Y. (2008). Simplemkl. *Journal of Machine Learning Research*, 9, 2491–2521.
- Rakotomamonjy, A., Bach, F., Canu, S., & Grandvalet, Y. (2007). More efficiency in multiple kernel learning. In *Proceedings of the 24th international conference on machine learning* (pp. 775–782).
- Schölkopf, B., & Smola, A. J. (2001). *Learning with kernels: Support vector machines, regularization, optimization, and beyond*. Cambridge, MA, USA: MIT Press.
- Shawe-Taylor, J., & Cristianini, N. (2004). *Kernel methods for pattern analysis*. Cambridge, UK: Cambridge University Press.
- Sloin, A., & Burshtein, D. (2008). Support vector training for improved hidden Markov modeling. *IEEE Transactions on Signal Processing*, 56(11), 172–188.
- Sonnenburg, S., Rätsch, G., Schäfer, C., & Schölkopf, B. (2006). Large scale multiple kernel learning. *Journal of Machine Learning Research*, 7, 1531–1565.
- Sturm, J. F. (1999). Using SeDuMi 1.02, a MATLAB toolbox for optimization over symmetric cones. *Optimization Methods and Software*, 11(1), 625–653.
- Szafranski, M., Grandvalet, Y., & Rakotomamonjy, A. (2008). Composite kernel learning. In *Proceedings of the 25th international conference on machine learning* (pp. 1040–1047).
- The MathWorks, Inc. (1998). *Statistics toolbox computation visualization programming for use with matlab user's guide*.
- Todd, M. J., Toh, K. C., & Tütüncü, R. H. (1998). On the nesterov-todd direction in semidefinite programming. *SIAM Journal on Optimization*, 8(3), 769–796.
- Tsang, I. W.-H., & Kwok, J. T.-Y. (2006). Efficient hyperkernel learning using second-order cone programming. *IEEE Transactions on Neural Networks*, 17(1), 48–58.
- Tsoi, A. C., & Tan, S. (1997). Recurrent neural networks: A constructive algorithm, and its properties. *Neurocomputing*, 15, 309–326.
- Vandenbergh, L., & Boyd, S. (1996). Semidefinite programming. *SIAM Review*, 38(1), 49–95.
- Wang, L. P. (2005). *Support vector machines: theory and application*. Berlin, Germany: Springer.
- Wang, Z., Chen, S., & Sun, T. (2008). MultiK-MHKS: A novel multiple kernel learning algorithm. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(2), 348–353.
- Wang, L. P., & Fu, J. J. (2005). *Data mining with computational intelligence*. Berlin, Germany: Springer.
- Ward, J. J., McGuffin, L. J., Buxton, B. F., & Jones, D. T. (2003). Secondary structure prediction with support vector machines. *Bioinformatics*, 19(13), 1650–1655.
- Xu, Z., Jin, R., King, I., & Lyu, M. (2008). An extended level method for efficient multiple kernel learning. *Advances in Neural Information Processing Systems* (Vol. 21). Cambridge, MA, USA: MIT Press.
- Yeh, C.-Y., Huang, C.-W., & Lee, S.-J. (2011). A multiple-kernel support vector regression approach for stock market price forecasting. *Expert Systems with Applications*, 38(3), 2177–2186.
- Yeh, C.-Y., Su, W.-P., & Lee, S.-J. (2011). Employing multiple-kernel support vector machines for counterfeit banknote recognition. *Applied Soft Computing*, 11(1), 1439–1447.